

HANDS-ON 4 [Intermediate]

Flask — App Structure, Routing, Jinja2 & Blueprints

Topics Covered:

Flask App Structure✓

Routing & URL Rules✓

Jinja2 Templates✓

Request & Response Objects✓

Blueprints for Modular Design✓

Flask Configuration✓

Flask is a lightweight, unopinionated micro-framework. You will rebuild the Course Management API backend in Flask, appreciating how Flask's minimal approach differs from Django's batteries-included philosophy.

SETUP

pip install flask flask-sqlalchemy flask-migrate then create app.py as the entry point

Task 1: Flask App Structure and Basic Routing

Goal: Set up a Flask application with proper structure and define routes.

Steps:

36.

Create a project folder flask_coursemanager/. Inside, create: app.py (entry point), config.py (configuration class), and a courses/ package with __init__.py and routes.py.

37.

In app.py, create the Flask app using the application factory pattern: def create_app(): app = Flask(__name__); ... return app. This pattern makes the app testable.

38.

In config.py, define a Config class with SQLALCHEMY_DATABASE_URI, SECRET_KEY, and DEBUG settings. Load config in create_app() using app.config.from_object(Config).

39.

In courses/routes.py, define a Blueprint: courses_bp = Blueprint('courses', __name__, url_prefix='/api/courses'). Add route handlers for GET / and POST /.

40.

Register the blueprint in create_app() with app.register_blueprint(courses_bp).

41.

Run the app and test that GET /api/courses/ returns a JSON response using jsonify({}).

Hint:

-

The application factory pattern (create_app function) is best practice in Flask — it avoids circular imports and makes testing easier.

-

Blueprints are Flask's way of organising routes into modules — similar to Django apps but lighter.

Expected Outcome: GET /api/courses/ returns an empty JSON array. POST returns a 201 with the created course. The Blueprint is registered correctly.

Task 2: Request Handling and JSON Responses

Goal: Handle incoming request data and return properly structured JSON responses.

Steps:

42.

In the POST / route, use request.get_json() to parse the request body.

Validate that required fields (name, code, credits) are present — return 400 with an error message if any are missing.

43.

Add a GET /<int:course_id>/ route, a PUT /<int:course_id>/ route, and a DELETE /<int:course_id>/ route in the blueprint.

44.

Create a helper function make_response_json(data, status_code) that always returns a consistent JSON envelope: {'status': 'success', 'data': data}.

45.

Add Flask error handlers for 404 and 500 using @app.errorhandler(404).

Return JSON error responses (not HTML) — APIs should never return HTML error pages.

46.

Use Postman to test all endpoints. Verify that missing fields return 400, unknown IDs return 404, and successful operations return 200 or 201.

Hint:

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

-

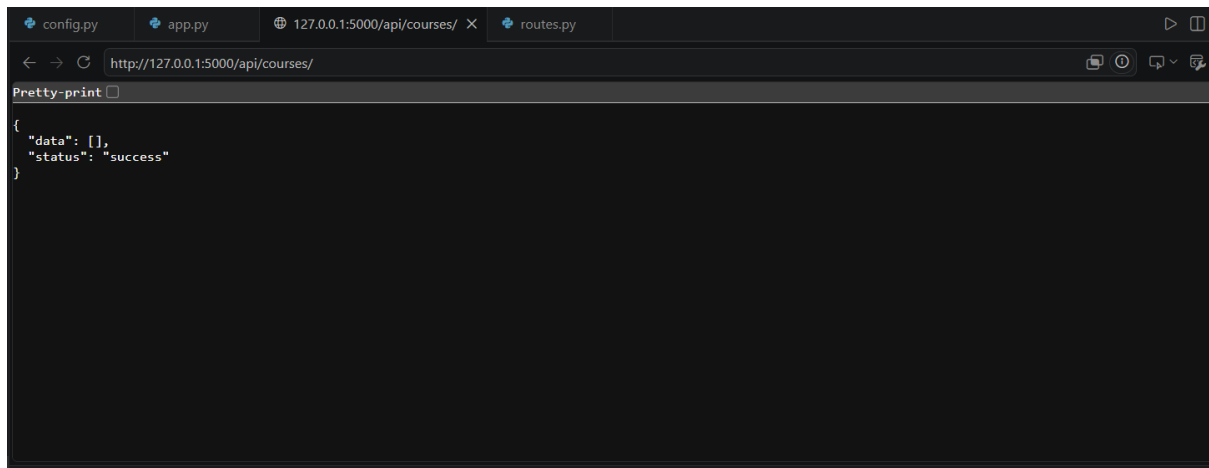
request.get_json() returns None if the Content-Type header is not application/json — always check for None before accessing keys.

-

Flask's default error responses are HTML. Register JSON error handlers so your API is consistent for all clients.

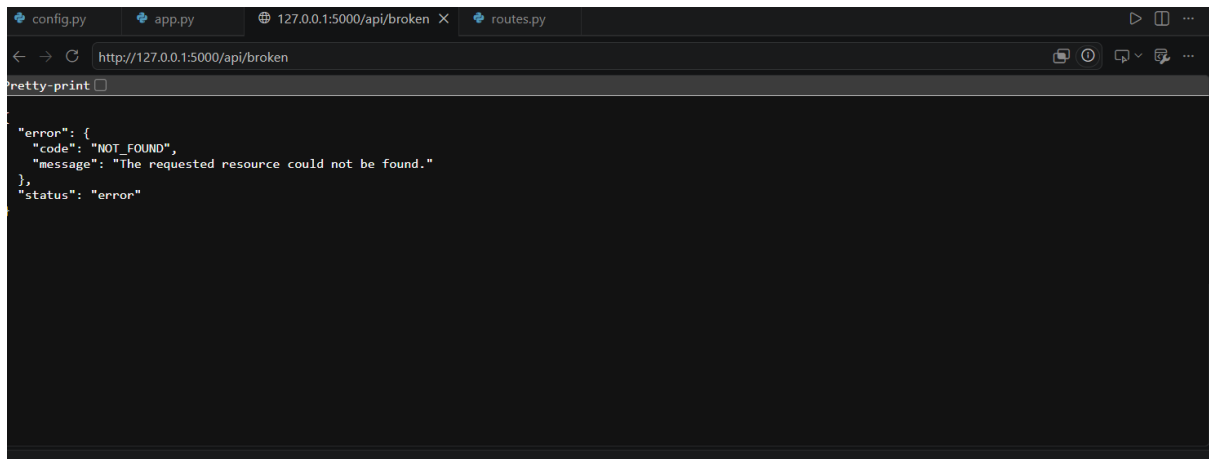
Expected Outcome: All endpoints return JSON. Missing fields trigger a 400 error with a descriptive message. Unknown IDs return 404.

Outcome:



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/api/courses/`. The browser's developer tools are open, showing the response of the GET request. The response is a JSON object with the following structure:

```
{  "data": [],  "status": "success"}
```



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/api/broken`. The browser's developer tools are open, showing the response of the GET request. The response is a JSON object indicating an error:

```
{  "error": {    "code": "NOT_FOUND",    "message": "The requested resource could not be found."  },  "status": "error"}
```